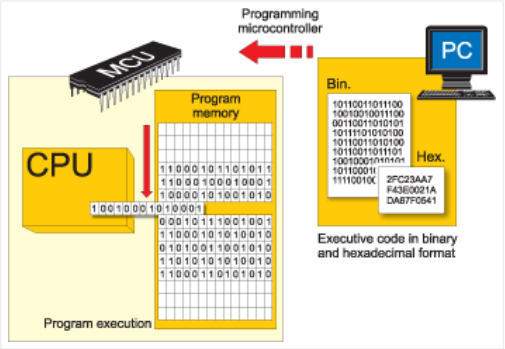
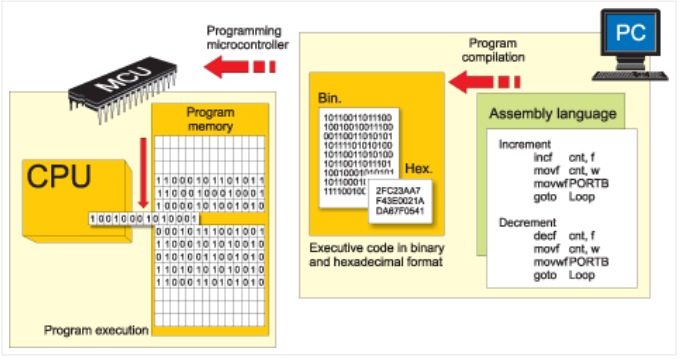
2.1 PROGRAMMING LANGUAGES

MIKROELEKTRONIKA

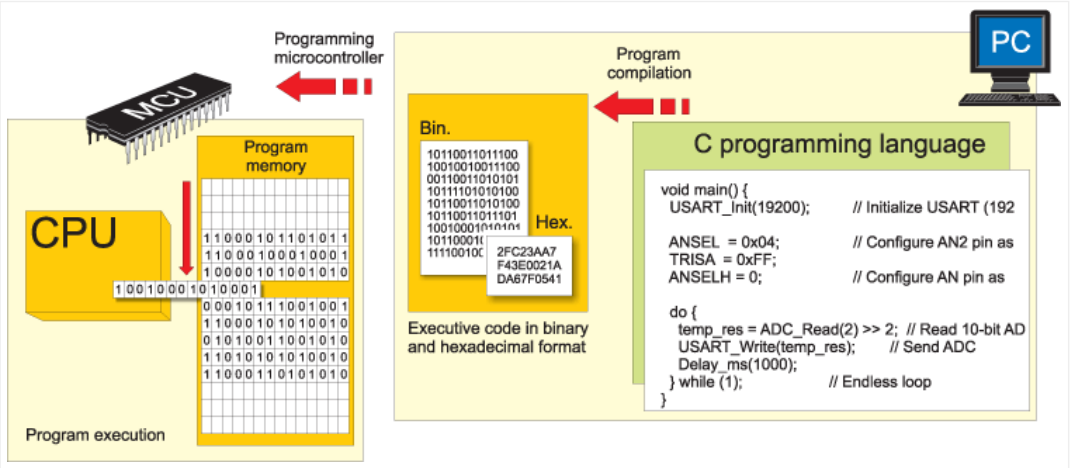


The microcontroller executes the program loaded in its Flash memory. This is the so called executable code comprised of seemingly meaningless sequence of zeros and ones. It is organized in 12-, 14- or 16-bit wide words, depending on the microcontroller's architecture. Every word is considered by the CPU as a command being executed during the operation of the microcontroller. For practical reasons, as it is much easier for us to deal with hexadecimal number system, the executable code is often represented as a sequence of hexadecimal numbers called a Hex code. It used to be written by the programmer. All instructions that the microcontroller can recognize are together called the Instruction set. As for PIC microcontrollers the programming words of which are comprised of 14 bits, the instruction set has 35 different instructions in total.

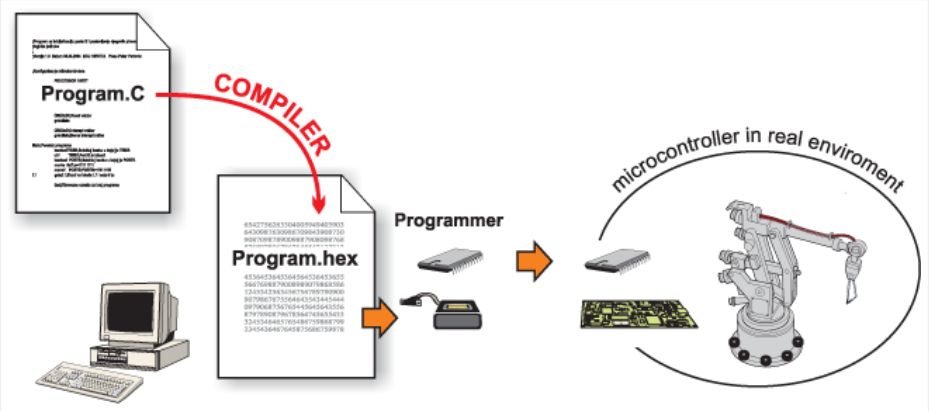


As the process of writing executable code was endlessly tiring, the first 'higher' programming language called assembly language was created. The truth is that it made the process of programming more complicated, but on the other hand the process of writing program stopped being a nightmare. Instructions in assembly language are represented in the form of meaningful abbreviations, and the process of their compiling into executable code is left over to a special program on a PC called compiler. The main advantage of this programming language is its simplicity, i.e. each program instruction corresponds to one memory location in the microcontroller. It enables a complete control of what is going on within the chip, thus making this language commonly used today.
However, programmers have always needed a programming language close to the language being used in everyday life. As a result, the higher programming languages have been created. One of them is C. The main advantageof these languages is simplicity of program writing. It is no longer possible to know exactly how each

command executes, but it is no longer of interest anyway. In case it is, a sequence written in assembly language can always be inserted in the program, thus enabling it.

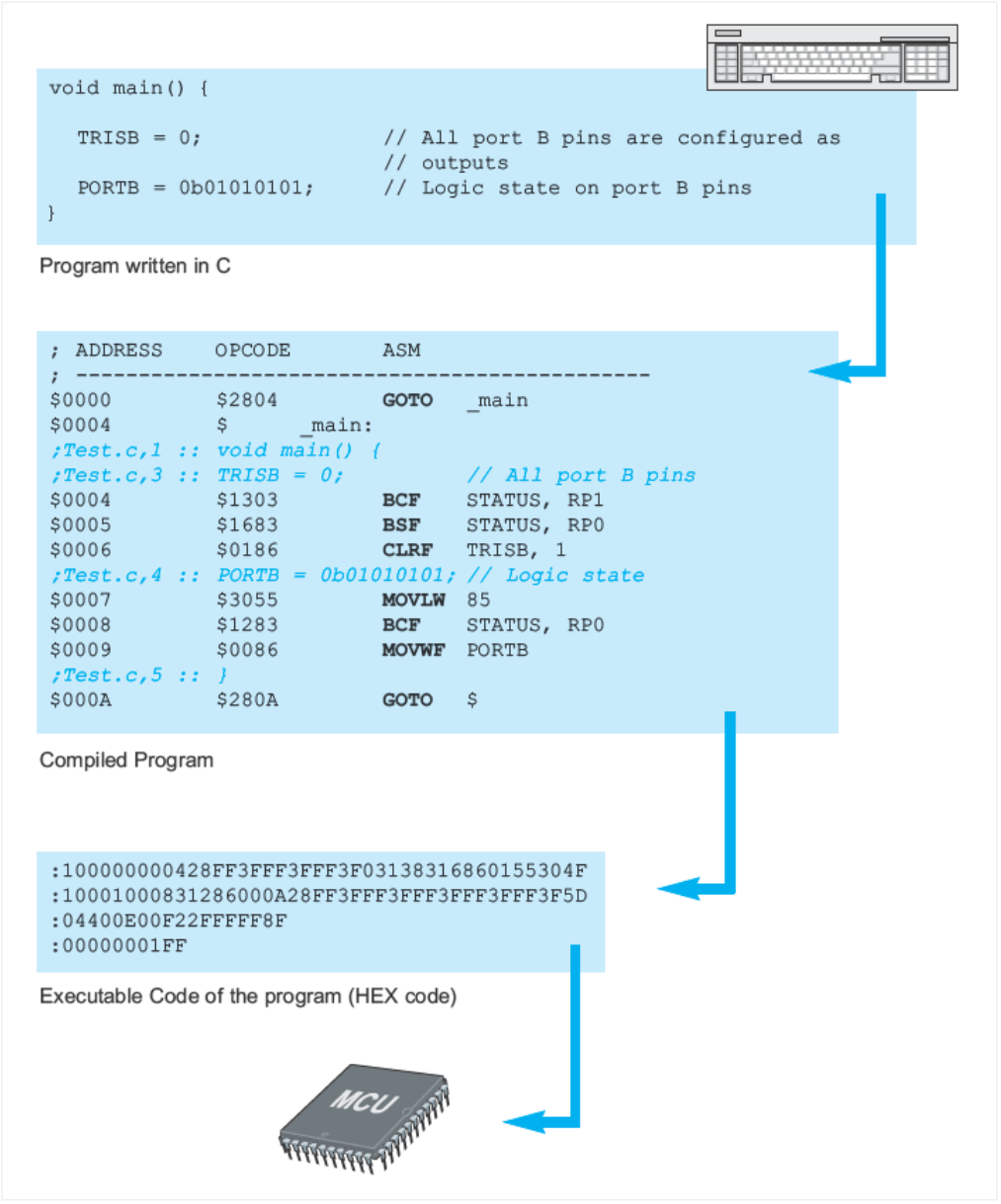


Similar to assembly language, a specialized program in a PC called compiler is in charge of compiling program into machine language. Unlike assembly compilers, these create an executable code which is not always the shortest possible.



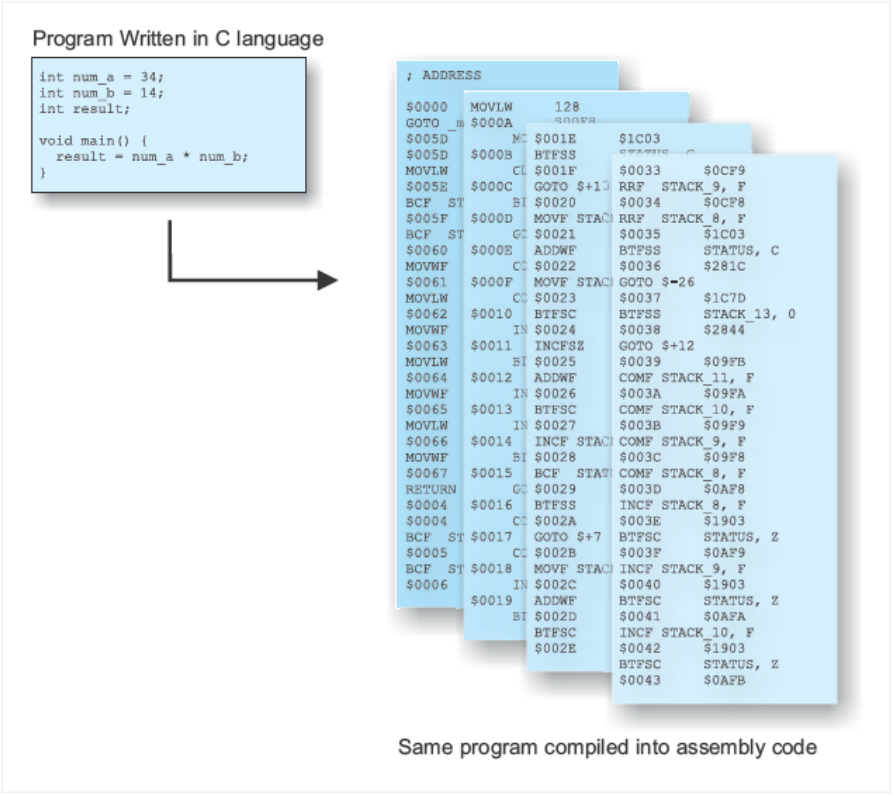
Figures above give a rough illustration of what is going on during the process of compiling the program from higher to lower programming language.

Here is an example of a simple program written in C language:



ADVANTAGES OF HIGHER PROGRAMMING LANGUAGES

If you have ever written a program for the microcontroller in assembly language, then you probably know that the RISC architecture lacks instructions. For example, there is no appropriate instruction for multiplying two numbers, but there is also no reason to be worried about it. Every problem has a solution and this one makes no exception thanks to mathematics which enable us to perform complex operations by breaking them into a number of simple ones. Concretely, multiplication can be easily substituted by successive addition (a x b = a + a + a + … + a). And here we are, just at the beginning of a very long story… Don't worry as far as the higher programming languages, such as C, are concerned because somebody has already solved this and many other similar problems for you. It will do to write a*b.

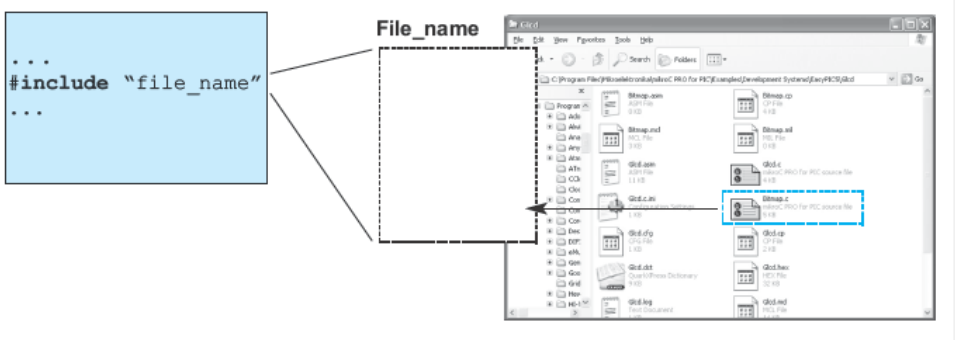


PREPROCESSOR

A preprocessor is an integral part of the C compiler and its function is to recognize and execute preprocessor instructions. These are special instructions which do not belong to C language, but are a part of software package coming with the compiler. Each preprocessor command starts with '#'. Prior to program compilation, C compiler activates the preprocessor which goes through the program in search for these signs. If any encountered, the preprocessor will simply replace them by another text which, depending on the type of command, can be a file contents or just a short sequence of characters. Then, the process of compilation may start. The preprocessor instructions can be anywhere in the source program, and refer only to the part of the program following their appearance up to the end of the program.

PREPROCESSOR DIRECTIVE # include

Many programs often repeat the same set of commands for several times. In order to speed up the process of writing a program, these commands and declarations are usually grouped in particular files that can easily be included in the program using this directive. To be more precise, the #include command imports text from another document, no matter what it is (commands, comments etc.), into the program.



PREPROCESSOR DIRECTIVE # define

The *#define* command provides macro expansion by replacing identifiers in the program by their values.

```
#define symbol sequence_of_characters
```

Example:

```
...
#define PI 3.14
...
```

As the use of any language is not limited to books and magazines only, this programming language is not closely related to any special type of computers, processors or operating systems. C language is actually a general-purpose language. However, exactly this fact can cause some problems during operation as C language slightly varies depending on its application (this could be compared to different dialects of one language).